

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
```

```
uploaded = files.upload()

video_name = list(uploaded.keys())[0]
print("Uploaded video:", video_name)
```

vlipsy-batm...rdX0rh.mp4

vlipsy-batman-the-animated-series-batman-pose-txrdX0rh.mp4(video/mp4) - 481027 bytes, last modified: 9/22/2026 - 100% done
 Saving vlipsy-batman-the-animated-series-batman-pose-txrdX0rh.mp4 to vlipsy-batman-the-animated-series-batman-pose-txrdX0rh.
 Uploaded video: vlipsy-batman-the-animated-series-batman-pose-txrdX0rh.mp4

```
cap = cv2.VideoCapture(video_name)

if not cap.isOpened():
    print("Error: Could not open video.")
else:
    print("Video opened successfully.")

fps = cap.get(cv2.CAP_PROP_FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

print("FPS:", fps)
print("Width:", width)
print("Height:", height)
print("Total Frames:", frame_count)
```

Video opened successfully.
 FPS: 30.0
 Width: 854
 Height: 480
 Total Frames: 273

```
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

ret, frame1 = cap.read()
ret, frame2 = cap.read()

if not ret:
    print("Could not read video frames.")
else:
    print("Frames successfully loaded.")

gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB))
plt.title("Frame 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB))
plt.title("Frame 2")
plt.axis("off")

plt.show()
```

Frames successfully loaded.

Frame 1



Frame 2



```
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

p0 = cv2.goodFeaturesToTrack(gray1, mask=None, **feature_params)

print("Number of detected feature points:", len(p0))
```

Number of detected feature points: 16

```
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03)
)

p1, st, err = cv2.calcOpticalFlowPyrLK(
    gray1,
    gray2,
    p0,
    None,
    **lk_params
)

good_new = p1[st == 1]
good_old = p0[st == 1]

print("Successfully tracked points:", len(good_new))
```

Successfully tracked points: 16

```
lk_output = frame2.copy()

for new, old in zip(good_new, good_old):
    x_new, y_new = new.ravel()
    x_old, y_old = old.ravel()

    cv2.arrowedLine(
        lk_output,
        (int(x_old), int(y_old)),
        (int(x_new), int(y_new)),
        (0, 255, 0),
        2,
        tipLength=0.3
    )

    cv2.circle(
        lk_output,
        (int(x_new), int(y_new)),
        4,
        (0, 0, 255),
        -1
    )

plt.figure(figsize=(12, 7))
plt.imshow(cv2.cvtColor(lk_output, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Sparse Optical Flow")
plt.axis("off")
plt.show()
```

Lucas-Kanade Sparse Optical Flow



```
flow = cv2.calcOpticalFlowFarneback(
    gray1,
    gray2,
    None,
    pyr_scale=0.5,
    levels=3,
    winsize=15,
    iterations=3,
    poly_n=5,
    poly_sigma=1.2,
    flags=0
)

print("Farneback dense optical flow calculated.")
```

Farneback dense optical flow calculated.

```
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

print("Maximum motion magnitude:", np.max(magnitude))
print("Average motion magnitude:", np.mean(magnitude))
```

Maximum motion magnitude: 42.09986
Average motion magnitude: 3.5297043

```
hsv = np.zeros_like(frame1)

hsv[..., 1] = 255

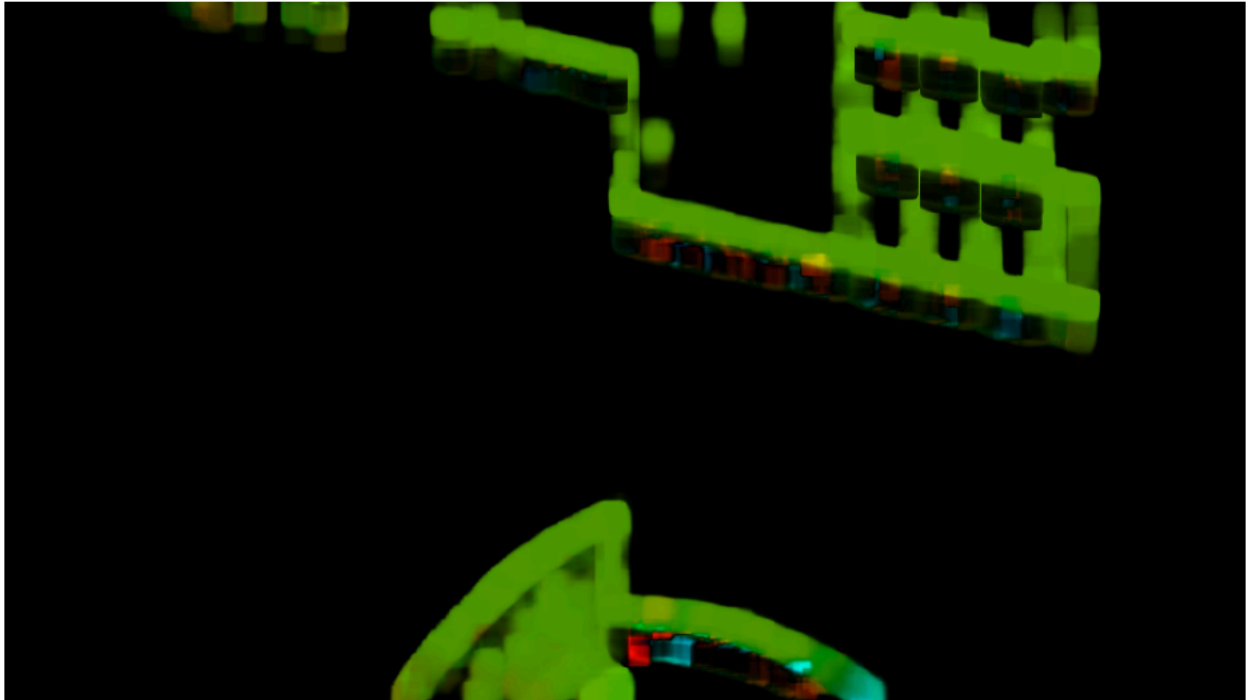
hsv[..., 0] = angle * 180 / np.pi / 2

hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

dense_flow = cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB)

plt.figure(figsize=(12, 7))
plt.imshow(dense_flow)
plt.title("Farneback Dense Optical Flow")
plt.axis("off")
plt.show()
```

Farneback Dense Optical Flow



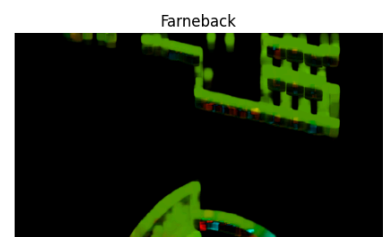
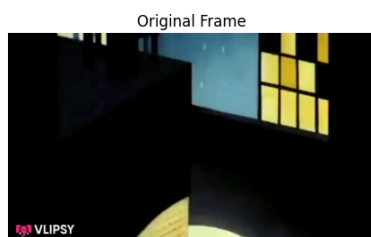
```
plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB))
plt.title("Original Frame")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(lk_output, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(dense_flow)
plt.title("Farneback")
plt.axis("off")

plt.show()
```



Step 7 — Comparison of Lucas-Kanade and Farneback Optical Flow

Parameter	Lucas-Kanade	Farneback
Type	Sparse Optical Flow	Dense Optical Flow
Motion Estimation	Selected feature points	Almost every pixel
Computational Cost	Lower	Higher
Speed	Generally faster	Generally slower
Output	Motion vectors and trajectories	Dense color-coded motion field
Accuracy	Good for selected features	Good for overall scene motion
Suitable For	Feature and object tracking	Complete scene-motion analysis

Lucas-Kanade is useful when specific feature points need to be tracked, while Farneback is useful when motion information from a large part of the scene is required.

Step 8 — Analysis of Optical Flow

Effect of Object Speed

Slow-moving objects can generally be tracked more accurately. Very fast movement can reduce the accuracy of optical flow because the displacement between consecutive frames becomes large.

Effect of Lighting Conditions

Changes in lighting, shadows, reflections, and sudden brightness changes can affect optical-flow estimation. This happens because optical-flow algorithms depend on changes in image intensity between frames.

Effect of Camera Motion

When the camera moves, stationary objects may also appear to move in the video. Therefore, optical flow may represent both actual object motion and camera-induced motion.

Observation

The accuracy of optical flow depends on object speed, lighting conditions, camera movement, image quality, and the amount of motion between consecutive frames.

Step 9 — Suitability for Real-World Applications

Lucas-Kanade Optical Flow

Lucas-Kanade is suitable for applications where selected features or objects need to be tracked.

Examples:

- Object tracking
- Feature tracking
- Gesture tracking
- Video stabilization

Farneback Optical Flow

Farneback is suitable when motion information from a large part of the scene is required.

Examples:

- Video surveillance
- Autonomous navigation
- Motion segmentation
- Scene-motion analysis

Overall

Lucas-Kanade is useful for tracking specific points efficiently, while Farneback provides a more detailed representation of motion throughout the scene.

Step 10 — Summary

Optical flow is an important computer vision technique used to estimate the apparent motion of objects, surfaces, or the camera between consecutive video frames.

In this experiment, Lucas-Kanade Sparse Optical Flow was used to track selected feature points, while Farneback Dense Optical Flow was used to estimate motion across almost every pixel of the image.

The experiment demonstrated that Lucas-Kanade has lower computational requirements and is useful for feature and object tracking. Farneback provides a more complete representation of scene motion and is useful for applications requiring dense motion information.

Optical flow plays an important role in dynamic scene analysis and motion-based computer vision systems such as autonomous driving, video surveillance, robotics, gesture recognition, sports analysis, and video stabilization.

Q1. What is Optical Flow? How is it used in computer vision?

Answer

Optical Flow is the apparent movement of objects, surfaces, or pixels between two consecutive frames of a video. It represents the direction and magnitude of motion in an image.

In computer vision, optical flow is used for object tracking, motion detection, video stabilization, gesture recognition, action recognition, robotics, and autonomous navigation.

Q2. Explain the working principle of the Lucas-Kanade Optical Flow algorithm.

Answer

Lucas-Kanade is a Sparse Optical Flow algorithm that tracks selected feature points between consecutive video frames.

The algorithm first detects important feature points in the first frame. It then considers a small window around each point and compares it with the corresponding region in the next frame. Using image gradients and the assumption that brightness remains approximately constant, it calculates the displacement of the feature point.

The movement is represented using motion vectors or arrows. Lucas-Kanade is commonly used for feature tracking and object tracking.

Q3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow?

Answer

Dense Optical Flow estimates the motion of almost every pixel in an image.

Sparse Optical Flow tracks only selected feature points, while Dense Optical Flow estimates motion over a large number of pixels.

Sparse Optical Flow	Dense Optical Flow
Tracks selected feature points	Tracks almost every pixel
Uses fewer points	Uses many pixels
Generally faster	Generally more computationally expensive
Example: Lucas-Kanade	Example: Farneback
Suitable for feature tracking	Suitable for complete scene-motion analysis

Q4. Compare the Lucas-Kanade and Farneback optical flow algorithms.

Answer

Parameter	Lucas-Kanade	Farneback
Type	Sparse	Dense
Tracks	Selected feature points	Almost every pixel
Computational Cost	Lower	Higher
Speed	Generally faster	Generally slower
Output	Motion vectors/trajectories	Dense color-coded motion field
Suitable For	Feature and object tracking	Scene-motion analysis

Lucas-Kanade is useful when specific features need to be tracked, whereas Farneback provides motion information across a larger portion of the image.

Q5. What assumptions are made while computing optical flow?

Answer

Optical flow generally works under the following assumptions:

1. Brightness constancy — the brightness of a moving point remains approximately constant between consecutive frames.
2. Small motion — the movement between two consecutive frames is relatively small.
3. Spatial consistency — nearby pixels generally have similar motion.
4. Smooth motion — motion is assumed to change gradually in nearby regions.

These assumptions help the algorithm estimate the movement of objects and pixels between frames.

Q6. What factors can affect the accuracy of optical flow estimation?

Answer

The accuracy of optical flow can be affected by several factors:

1. Fast movement of objects
2. Sudden changes in lighting
3. Shadows and reflections
4. Camera movement
5. Motion blur
6. Low-quality or noisy images
7. Occlusion of objects
8. Low frame rate
9. Low-texture regions
10. Large movement between consecutive frames

Therefore, clear images, sufficient frame rate, and stable lighting generally help improve optical-flow estimation.

Q7. Mention five real-world applications of optical flow in computer vision.

Answer

Five important applications of optical flow are:

1. Autonomous driving — estimating the movement of vehicles, pedestrians, and the surrounding environment.
2. Video surveillance — detecting and tracking moving people or objects.
3. Action recognition — analyzing human movements and gestures.
4. Robotics — helping robots understand movement in their surroundings.
5. Sports analysis — tracking players and analyzing their movements.

Other applications include video stabilization, drone navigation, motion detection, and augmented reality.

Q8. Why are grayscale images generally used for optical flow computation?

Answer

Grayscale images are generally used because optical flow mainly depends on brightness or intensity changes and image gradients rather than color information.

Using grayscale images:

- Reduces the amount of data to process.
- Makes calculations faster.
- Simplifies gradient calculations.
- Reduces computational complexity.

Therefore, video frames are commonly converted from color to grayscale before optical-flow computation.

Q9. What are the limitations of optical flow algorithms in real-world environments?

Answer

Some important limitations of optical flow algorithms are:

1. Very fast-moving objects can be difficult to track.
2. Changes in lighting can produce incorrect motion estimates.
3. Camera movement can be confused with object movement.

4. Occlusion can cause feature tracking to fail.
5. Motion blur can reduce accuracy.
6. Low-texture regions may not provide enough information for tracking.
7. Dense optical flow can require significant computational resources.
8. The algorithms depend on assumptions such as brightness constancy and smooth motion.

Q10. How does optical flow contribute to applications such as autonomous driving, video surveillance, and action recognition?

Answer

Optical flow provides information about the direction and magnitude of motion in video sequences.

Autonomous Driving

Optical flow can help estimate the movement of vehicles, pedestrians, and the surrounding environment. It can contribute to